

understanding mup: is spectral condition for Feature Orthogonality
mup is based off the following assertion, as a function of
width of the network n ---

A1: The activations at initialization should remain $\mathcal{O}(1)$

A2: After one gradient step, the change in activation should
be $\mathcal{O}(1)$

Deriving mup: { condition A1 method } { condition A2 method } $\Rightarrow$ { zero-shot hyper-param }
{ transfer across }
{ different sized networks }

What is mup? Max updated param. is a reparameterization
scheme by Google and Seemford to ensure training dynamic
remain stable as network width $n \rightarrow$ infinite

How? scaling ^{two} conditions on weights and learning rate
so that activations and gradients do not blow up
or vanish when scaling model width.

$\Rightarrow$ Condition #1: stable forward activation scaling $\Rightarrow$ ensure
that the hidden activations h_i at each layer stay of
order $\mathcal{O}(1)$ as width n_i increase.

$\Rightarrow$ Condition #2: stable backward gradient scaling $\Rightarrow$ ensure updates
 Δh_i and parameter updates Δw remain of the
same scale ($\mathcal{O}(\sqrt{n_i})$), independent of width.

condition #1: control initialization (keep activations stable)
condition #2: control learning rate and update scaling (keeps training dynamic
stable)

Scale training Dynamics: keep gradients + learning rate from
O(1): same scale-level changes explosion and or vanishing

mup: control activations and gradients via W and dW .

initialization: set to $O(\frac{1}{\sqrt{n_{l-1}}} \min(1, \sqrt{\frac{n_l}{n_{l-1}}}))$

learning rate: set to $\frac{n_l}{n_{l-1}}$ (for Adam $\frac{1}{n_{l-1}}$)

$\Rightarrow$ standard parameterizations - these set:

activation initialization: set to $\frac{1}{\sqrt{n_{l-1}}}$

learning rates: set to $O(1)$

Constant magnitude
vanishing / unexploded
learning rate
constant scale
order

$1/n$: vanishes

n^2 : grows $\Rightarrow$ mup is a scaling procedure for hyperparameters
As a function of network width

Scaling protocol in network: only width scaling, keep LR constant

mup is robust to

- Activations - SwiGLU and relu.
- Batch sizes - small / large
- RMS Norm
- Optimizers
- Regularizers

mup is not robust to:

- weight decay
- exotic optimizers